

AI 하는 사람을 위한 Vercel 초간단

AI 하는 사람을 위한 Vercel 초간단 — 목차

나두Ai의 『AI 하는 사람을 위한 Vercel 초간단』은 AI·코딩을 막 시작한 분이 **Vercel 계정 → GitHub 연결 → 첫 배포 → Preview/Production → 환경변수 → (맞보기) 도메인**까지 화면을 따라가며 한 바퀴 돌리게 돕는 초보용 전자책입니다.

GitHub Actions 심화, Edge 미들웨어 심화, 대규모 모노레포, 결제·쇼핑몰 구축은 다루지 않습니다.

무료 PDF (배포 후 확정): [tinalinkeom /ebook](#) 계열

브랜드: 나두Ai

대상: AI·코딩 입문자 · GitHub는 조금 아는 분에게 더 쉬움

제공: 무료 · 개인 학습용 · 무단 재판매 금지 · Vercel 공식 교육 아님

전체 목차

장	제목	예상
1장	이 책으로 할 수 있는 것 / 안 다루는 것	3~4p
2장	Vercel 계정 만들기	3~4p
3장	GitHub 연결 · 프로젝트 만들기	4~5p
4장	첫 배포	4~5p
5장	Preview vs Production	3~4p
6장	환경변수(.env)	4~5p
7장	커스텀 도메인 맞보기	3~4p

장	제목	예상
8장	자주 막히는 장면 Q&A	3~4p
9장	다음에 연습할 것	2~3p
부록 A	나두식 따라하기 Lab	별도
부록 B	가벼운 템플릿 배포 Lab	별도
고지	고지·법적 주의	별도

나두Ai · 화면 따라하기용

1장. 이 책으로 할 수 있는 것 / 안 다루는 것

AI 도구로 코드·화면을 만들어 본 적이 있다면, 그 감각이면 충분합니다.

이 책이 메우는 빈칸은 **웹에 올려 주소로 여는 한 바퀴**뿐입니다.

ChatGPT·Cursor·Claude 등으로 파일을 받아 본 분은 많지만, “그래서 URL은 어디서 생기나?”에서 멈추는 경우가 많습니다. 그 빈칸이 Vercel입니다.

vercel.com 화면 문구는 가끔 바뀝니다. 버튼 글자가 조금 달라도 **비슷한 문구**면 그 버튼을 누르면 됩니다. (이 안내는 이 책에서 여기만 적습니다.)

1. 끝까지 따라가면 생기는 일

1. **Vercel** 계정을 만들고 로그인한다
2. **GitHub**를 연결한다
3. 저장소(또는 템플릿)로 **프로젝트**를 만든다
4. 첫 배포 후 주소(*.vercel.app)로 열어 본다
5. **Preview**와 **Production** 차이를 구분한다
6. (이어지는 장) **환경변수**를 넣고, **도메인**을 맞본다

계정 → **GitHub 연결** → 프로젝트 → 배포 → **Preview/Production** → (환경변수 · 도메인)
이 한 바퀴를 한 번 완주한다.

한 바퀴만 돌면 “푸시했더니 사이트가 갱신된다”는 감각이 생깁니다.

그다음부터는 AI가 준 코드도 **같은 파이프**에 흘려 넣으면 됩니다.

화면 참고: 책 전체 흐름 — 계정 → GitHub → 프로젝트 → 배포 → Preview/Prod → env → 도메인

2. 다루는 것

주제	의미	어디서
계정	vercel.com 가입·로그인	브라우저
GitHub 연결	저장소를 Vercel이 읽게 허용	vercel.com
프로젝트	배포 단위 하나 만들기	vercel.com
배포(Deploy)	빌드 후 인터넷 주소로 공개	vercel.com
Preview / Production	미리보기 주소 vs 본편 주소	Deployments
환경변수	키·비밀값을 서버 쪽에만 두기	Project Settings
도메인 맛보기	내 이름으로 붙이는 첫걸음	Domains

용어 한 줄

- **Vercel** — GitHub 등과 연결해 웹 앱을 빌드·배포하는 서비스 (<https://vercel.com>)
- **배포(Deploy)** — 코드를 서버에서 빌드하고 URL로 열리게 하는 일
- **프로젝트(Project)** — Vercel 안의 배포 단위 하나 (보통 GitHub 저장소 1개와 짝)
- **Preview** — 브랜치·PR용 미리보기 주소
- **Production** — 기본 브랜치가 가리키는 본편 배포
- **환경변수(Environment Variables)** — API 키 등 코드에 직접 박지 않는 설정값
- **도메인(Domain)** — [네이름.com](#) 처럼 붙이는 주소 (7장에서 맛보기)

프레임워크 이름은 몰라도 따라갈 수 있습니다. 화면에 **Next.js**가 보이면 “웹 앱 종류 하나” 정도로만 이해하면 됩니다.

3. Vercel vs GitHub (한 줄씩)

	하는 일
GitHub	코드를 올리고·보관·제안(PR) 하는 곳
Vercel	그 코드를 빌드해서 웹 주소로 열어 주는 곳

GitHub에 푸시했다고 바로 사이트 주소가 생기지는 않습니다.

Vercel을 연결하면 푸시 → 자동 배포 흐름이 열립니다.

비유하면 GitHub는 **창고·원고 보관소**, Vercel은 **인쇄해서 거리에 내거는 가게 간판**에 가깝습니다.

창고에만 두면 방문자는 못 보고, 간판만 있으면 고칠 원본이 없습니다. 둘을 잇는 것이 이 책의 핵심입니다.

이 책은 **Vercel 화면 따라가기**가 중심입니다. GitHub 계정·저장소·푸시·PR 기초는 『깃허브 초간단』 쪽을 전제로 짧게만 짚습니다.

GitHub가 전혀 없으면 2장에서 **Continue with GitHub**로 계정을 같이 준비하는 편이 덜 헛갈립니다.

4. 안 다루는 것

안 다룸	이유
GitHub Actions 심화	CI YAML·매트릭스 등 — 배포 한 바퀴 다음에
Edge 미들웨어 심화	라우팅·인증 미들웨어 설계
대규모 모노레포	워크스페이스·권한 규칙이 다름
결제·쇼핑몰 구축	Stripe·장바구니 전체 설계
팀·Enterprise 고급	SSO·조직 정책·유료 한도 심화

안 다름	이유
로컬 CLI 심화	<code>vercel</code> 명령 고급 옵션·스크립트

지금은 **한 바퀴 완주**가 목표입니다. 결제·쇼핑몰·Actions까지 할 필요 없습니다.
막히는 장면은 8장 Q&A에서 짧게 모읍니다.

5. 누가 읽으면 맞나

- AI로 프론트·랜딩·Next 비슷한 걸 받아 봤는데, **어디에 올리지**가 막힌다
- GitHub는 조금 아는데, **배포·URL·Preview**가 낯설다
- 혼자라도 **내 프로젝트 URL**을 만들어 보고 싶다
- “데모 링크 하나”가 필요해서 Vercel 이름을 검색해 본 적이 있다

이미 매일 Vercel Production을 굴리는 분에게는 복습용 수준입니다.
서버를 직접 임대·SSH로 운영하는 방법은 이 책 범위 밖입니다.

6. 준비물

꼭 필요: 인터넷 · 웹 브라우저 · 이메일 · (권장) **GitHub 계정** · 따라할 시간(장마다 나눠도 됨)

있으면 편함: 본인 GitHub 저장소 하나, 또는 Vercel **템플릿**으로 시작

기본 경로: 브라우저 + `https://vercel.com` · 로컬 `vercel` CLI 심화는 제외

휴대폰 브라우저만으로도 가입·Visit은 가능하지만, 화면이 좁아 Import·Settings가 불편할 수 있습니다. **PC·노트북 브라우저**를 권장합니다.

화면 참고: 준비물 — 인터넷 / 브라우저 / 이메일 / GitHub 계정

7. 읽는 법

1. 번호 매긴 **단계**를 그대로 따른다

2. 「여기서 막히면」 은 막혔을 때만 보면 됨

3. *화면 참고: ...* 는 캡처 자리 — 문구가 조금 달라도 같은 위치면 된다

연습용은 **본인이 만든 저장소·템플릿 프로젝트**에서 하세요.

중요한 회사·님의 Production에는 바로 적용하지 마세요.

여기서 막히면

“실서비스가 망가질까 봐 무섭다” → **연습용 프로젝트**를 새로 만드세요. 이름에 practice · lab 을 넣어도 됩니다.

8. 영어 버튼 (그 장에서 누르는 것만)

- **Sign Up / Log In** = 가입·로그인 · **Add New...** / **Import** = 가져오기
- **Deploy** = 배포 · **Visit** = 주소 열기 · **Redeploy** = 다시 배포
- **Preview / Production** = 미리보기 / 본편
- **Environment Variables** = 환경변수 · **Domains** = 도메인

전부 외울 필요 없습니다. 그 장에서 누르는 것만 보면 됩니다.

9. 안전 (짧게)

- **비밀번호·토큰·API 키**는 책·채팅·캡처·Public 저장소에 적지 마세요. (이후 장에서 반복하지 않습니다.)
- 연습은 **본인 계정·본인 프로젝트**에서. 남의 프로젝트에 함부로 배포하지 마세요.
- 이 책은 법률·세금·결제 안내가 아니며, **Vercel 공식 교육이 아닙니다**. 무료 개인 학습용입니다.

다음

장	할 일
2장	Vercel 계정
3장	GitHub 연결 · 프로젝트
4장	첫 배포
5장	Preview vs Production
6~7장	환경변수 · 도메인 맛보기
8~9장	막힘 Q&A · 혼자 연습

계정이 이미 있으면 1장의 범위만 확인하고 3장부터 가도 됩니다.

GitHub만 있고 Vercel이 없으면 **2장**부터 순서대로 가면 됩니다.

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

2장. Vercel 계정 만들기

브라우저만 열고 `https://vercel.com` 에서 **가입** → **로그인** → **대시보드**까지 합니다.

로컬 CLI·VS Code·도메인 구매는 이 장에서 필요 없습니다.

끝나면 “Vercel에 들어왔다”는 감각만 있으면 됩니다. 프로젝트 만들기는 3장입니다.

1. 시작 전

1. 인터넷 · 웹 브라우저 · 열어 볼 수 있는 **이메일**
2. (권장) **GitHub 계정** — Continue with GitHub로 한 번에 이어지기 쉽습니다
3. 비밀번호·토큰은 **안전한 곳**(비밀번호 관리 앱·종이 등)에만 두세요. **책·채팅·캡처에 적지 마세요.**

이미 Vercel 계정이 있으면 가입 절은 건너뛰고 **대시보드(5~6절)** 만 보면 됩니다.

GitHub 계정이 없다면 가입 전에 `github.com`에서 계정을 먼저 만들어 두는 편이 이후 장이 수월합니다.

여기서 막히면

예전에 가입한 것 같으면 새 가입 전에 **Log In**으로 이메일·GitHub 로그인을 먼저 시도하세요.
Forgot password / 이메일 재설정이 보이면 그걸 우선합니다.

화면 참고: *vercel.com — Sign Up / Log In* 부근

2. vercel.com 열기

1. 주소창에 `https://vercel.com` 입력
2. 상단 **Sign Up** 또는 **Log In** 클릭
3. 개인 학습은 **Hobby(무료 티어)** 기준으로 충분합니다. Pro·팀·Enterprise 안내는 지금은 무시해도 됩니다.

회사 메일 필수는 아닙니다. **임시(일회용) 메일**은 인증·복구가 막힐 수 있어 쓰지 마세요.
한국어·영어 UI가 섞여 보여도 버튼 위치만 같으면 됩니다.

화면 참고: *Sign Up 선택 화면 — GitHub / 이메일*

3. 가입 — GitHub로 하기 (권장)

초보 기본 경로입니다. 3장에서 저장소 Import할 때도 연결이 이미 잡혀 있으면 덜 헛갈립니다.

1. **Continue with GitHub** (문구가 Sign up with GitHub 등이어도 같은 뜻)
2. GitHub 로그인 창이 뜨면 Username/이메일 + 비밀번호
3. 이미 GitHub에 로그인돼 있으면 계정 선택만 나올 수 있습니다
4. Vercel이 요청하는 **Authorize / 권한 허용** 확인 후 승인
5. vercel.com으로 돌아오면 가입·로그인에 가깝습니다

권한 화면에서 저장소 범위(모든 저장소 / 일부만)를 고를 수 있습니다.

연습만이면 **본인 연습용 저장소만** 허용해도 됩니다. 나중에 GitHub App 설정에서 다시 조정할 수 있는 경우가 많습니다.

Organization(회사·동아리) 저장소를 지금 당장 쓸 계획이 없으면, 개인 계정 저장소만 선택하세요.

권한이 커 보여도 “배포 연습용”으로만 쓰면 됩니다. **토큰 문자열을 복사해 어딘가에 붙여 넣을 필요는 없습니다.**

화면 참고: *Continue with GitHub → Authorize · 저장소 범위*

4. 가입 — 이메일로 하기 (선택)

GitHub 없이 시작해도 됩니다. 다만 3장에서 **GitHub 연결을 따로** 해야 배포가 이어집니다.

1. 이메일 주소로 Sign Up
2. 비밀번호 설정 — 다른 사이트와 같은 비밀번호 재사용은 피하세요
3. 메일함에서 인증(verify / confirm) 링크·코드 처리

4. 다시 vercel.com에 로그인

메일이 없으면: 스팸·프로모션함 → 주소 오타 → Resend.

로그인만 됐고 인증만 남았으면 화면 배너의 Resend / 계정 Settings의 이메일 항목을 따릅니다.

이메일 가입 후에도 나중에 **Continue with GitHub**로 계정을 연결할 수 있는 경우가 많습니다.

가능하면 이 장에서 GitHub까지 붙여 두는 편이 3장이 짧아집니다.

화면 참고: 이메일 가입 · Verify 안내

5. 로그인 확인 · 대시보드

로그인되면 보통 **Dashboard**(프로젝트 목록)로 들어갑니다.

주소가 `vercel.com/대시보드경로` 형태로 바뀌고, 마케팅 랜딩보다 **프로젝트 카드·빈 목록**이 보이면 성공에 가깝습니다.

확인할 것:

1. 왼쪽·상단에 **Dashboard / Projects** 비슷한 메뉴
2. 오른쪽 위 **아바타·계정 메뉴** (Sign Up 대신)
3. 아직 프로젝트가 없으면 빈 목록 + **Add New...** / **Import Project** 안내

풀렀다면: `https://vercel.com/login` → Log In → GitHub 또는 이메일.

공용 PC·카페 컴퓨터에서는 실습 후 계정 메뉴에서 **Log Out**을 꼭 합니다.

화면 참고: Dashboard — 프로젝트 비어 있음 · Add New

5-1. 대시보드에서 지금 누르지 말 것

눈에 띄는 버튼이 많아도 이 장 목표는 **로그인·목록 확인**까지입니다.

- **Add New... / Import** — 3장에서
- **Domains / Buy Domain** — 7장 맛보기
- **Upgrade / Billing** — 연습 Hobby면 달기

- **Integrations 마켓** — 한 바퀴 후

빈 Projects 화면이 “잘못됐다”는 뜻이 아닙니다.

계정이 준비됐다는 뜻에 가깝습니다. 다음 장에서 카드를 하나 채웁니다.

브라우저에 비밀번호를 저장할지 묻는 창이 뜨면, **본인 PC**에서만 저장하세요.

공용 PC면 저장하지 말고 Log Out합니다.

6. 계정·팀 이름만 훑기

가입 직후 **팀(Team) 이름**이나 사용자명을 묻는 마법사가 뜰 수 있습니다.

1. 연습용이면 짧은 영문 이름이면 충분합니다 (예: `my-nadoo-lab`)
2. 얼굴·실명·바이오 공개는 필수가 아닙니다
3. Skip / 나중에 / Continue가 보이면, 한 바퀴를 먼저 완주해도 됩니다

Billing·카드 등록·유료 플랜 비교 안내는 **이 장에서 하지 않습니다.**

Hobby로 연습 배포는 보통 카드 없이 가능합니다. 유료 전환·크레딧 구매 화면이 뜨면 닫고 대시보드로 돌아오세요.

팀 초대를 받았다면, 연습은 **본인이 만든 Hobby** 쪽에서 하는 편이 권한이 단순합니다.

남의 팀 Production을 건드리지 마세요.

화면 참고: 온보딩 — 팀/사용자명 (선택)

6-1. 로그인만 반복 연습

1. 아바타 → **Log Out** (Sign out 문구여도 같음)
2. `https://vercel.com/login` → 다시 Log In
3. GitHub로 가입했다면 **Continue with GitHub**가 가장 빠릅니다
4. Dashboard Projects가 다시 보이면 충분합니다

브라우저가 이전 세션을 기억해 자동 로그인이 되면, 공용 PC가 아닐 때는 그대로 써도 됩니다.

다만 “내가 어떤 계정으로 들어왔는지”는 아바타·Settings의 이메일·GitHub 연결로 한 번 확인하세요.

2FA·보안키를 GitHub에 켜 두었다면 Vercel GitHub 로그인 때도 그 확인이 필요할 수 있습니다.
이 책은 2FA 설정을 강제하지 않습니다. 이미 켜 두었다면 안내 그대로 따르면 됩니다.

7. 이 장에서 안 함

프로젝트 Import(3장) · 첫 Deploy(4장) · Preview/Production(5장) · 도메인 · 환경변수 ·
vercel CLI · 유료 플랜 결제 · 팀원 초대 심화

기본 경로: 브라우저 + <https://vercel.com> (+ 가능하면 GitHub 로그인)

짧게 정리

vercel.com → **Sign Up** → (권장) **Continue with GitHub** → Authorize → **Dashboard**
확인 → Log Out/In 한 번 연습.

다음 **3장**에서 GitHub를 연결하고 **프로젝트**를 만듭니다. Add New가 보여도 급히 Deploy만
연타하지 말고, 3장 순서를 따릅니다.

화면 참고: 3장 예고 — *Add New...* / *Import Git Repository*

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

3장. GitHub 연결 · 프로젝트 만들기

로그인된 상태에서 **GitHub**를 연결하고, **프로젝트 하나**를 만듭니다.

배포 버튼을 눌러도 되지만, **결과 확인·Redeploy**는 4장에서 천천히 봅니다.

이 장의 성공 기준은 Dashboard **Projects**에 카드가 하나 보이는 것입니다.

1. 이 장에서 만드는 것

프로젝트 = Vercel 안의 배포 단위 하나 (대개 GitHub 저장소 1개와 짝)

시작하는 방법은 두 가지입니다. **하나만** 하면 됩니다.

방법	언제
템플릿	GitHub에 올릴 앱이 아직 없을 때 (Next.js 템플릿 OK)
기존 저장소 Import	이미 GitHub에 저장소가 있을 때

둘 다 결국 **GitHub ↔ Vercel**이 연결되어야 이후 푸시 배포가 편합니다.

템플릿을 고르면 Vercel이 GitHub에 새 저장소를 만들어 주는 흐름이 많아, GitHub가 서툰 분에게도 무난합니다.

화면 참고: *Add New — Project / Import*

2. GitHub 연결 확인

1. `https://vercel.com` — Dashboard 로그인
2. 아바타 → **Settings** → **Login Connections / Git** 비슷한 메뉴
또는 Import 화면에서 **Connect GitHub**가 보이면 그걸로도 됩니다
3. GitHub가 **Connected**인지 확인. 아니면 **Connect** → GitHub Authorize

저장소가 목록에 안 보이면 아래를 순서대로 봅니다.

- GitHub App 설치 범위에 **그 저장소**가 포함됐는지
- **Only select repositories**로 막아 두지 않았는지
- Organization 저장소면 Org 관리자 승인·Install이 필요한지

이 책은 **본인 개인 계정 저장소**를 기본으로 합니다.

회사 Org는 권한이 복잡해져 초보 한 바퀴와 맞지 않는 경우가 많습니다.

여기서 막히면

2장에서 이메일만으로 가입했다면 여기서 GitHub Connect를 꼭 끝내세요.

Authorize 창이 바로 닫히면 팝업 차단을 끄고 다시 Connect합니다.

화면 참고: *Connect GitHub · Authorize · 저장소 접근 범위*

3. 템플릿으로 만들기 (Next.js OK)

저장소가 없어도 됩니다.

1. Dashboard → **Add New...** → **Project**
2. **Clone Template** / 템플릿 갤러리에서 **Next.js** (또는 Beginner용 공식 템플릿) 선택
3. GitHub **저장소 이름** 입력 (예: `hello-vercel`)
4. Private/Public — 연습·확신이 없으면 **Private**
5. **Create / Deploy** 계열 버튼

이름이 중복이면 `hello-vercel-2` 처럼 바꿉니다.

영문·숫자·하이픈(-)이면 충분하고, 공백은 피합니다.

Framework Preset이 Next.js로 잡혀 있으면 초보에게 충분합니다.

Build Command·Output Directory를 지금 손으로 고칠 필요 없습니다. 기본값을 유지하세요.

템플릿 Deploy를 누르면 GitHub에 저장소가 생기고 Vercel 빌드가 시작되는 경우가 많습니다. 빌드 로그 읽기는 4장으로 미뤄도 됩니다.

4. 기존 GitHub 저장소 Import

이미 연습용 저장소가 있다면 템플릿 대신 이 절만 하면 됩니다.

1. Dashboard → **Add New...** → **Project**
2. **Import Git Repository** 목록에서 저장소 선택
3. 안 보이면 **Adjust GitHub App Permissions** / Configure → 저장소 체크 후 저장 → vercel.com으로 돌아와 새로고침
4. **Import** 클릭

Import 직전 설정 (초보 기본값)

항목	초보 선택
Project Name	저장소 이름 그대로 OK
Framework Preset	자동 감지값 유지 (Next.js면 Next.js)
Root Directory	모노레포가 아니면 비움 또는 <code>.</code>
Build · Output	기본값 (지금은 건드리지 않음)
Environment Variables	이 장에서는 비움 → 6장

Deploy를 누르면 빌드가 시작됩니다. 성공·실패 화면 읽기는 **4장**.

AI로 받은 코드가 “완전 빈 폴더”이거나 `package.json` 이 없으면 빌드가 실패할 수 있습니다. 그럴 때는 템플릿으로 먼저 한 바퀴를 돌린 뒤, 같은 방식으로 Import를 다시 시도하세요.

화면 참고: Import Git Repository — 저장소 선택 · Deploy

4-1. Import 전에 저장소만 잠깐 확인

기존 저장소를 쓸 때 GitHub에서 아래만 눈으로 봅니다.

1. 저장소가 **본인 계정** 아래인지 (남의 fork에 배포 권한이 없을 수 있음)
2. 기본 브랜치에 파일이 있는지 (완전 빈 저장소면 템플릿이 더 빠름)
3. README·소스에 **API 키 문자열**이 없는지

Vercel은 GitHub의 그 브랜치를 읽어 빌드합니다.

로컬에만 있고 푸시하지 않은 변경은 배포에 안 들어갑니다.

“내 PC에서는 되는데 Vercel만 실패”면, **푸시 여부**를 먼저 의심하세요.

Framework를 수동으로 잔뜩 바꾸기보다, 자동 감지 + 기본 Build를 유지하는 편이 초보에게 안전합니다.

빌드가 실패한 뒤에야 Settings에서 Preset을 손보는 순서가 덜 헛갈립니다.

5. 프로젝트가 생겼는지 확인

Deploy를 눌렀든, 생성만 됐든 Dashboard **Projects**에 카드가 보이면 성공에 가깝습니다.

확인할 것:

1. 프로젝트 **이름**
2. 연결된 **Git 저장소** 이름·아이콘
3. (이미 배포됐다면) `*.vercel.app` 주소 힌트

프로젝트 페이지 상단에 **Deployments / Settings / Domains** 탭이 보이면 이후 장에서 씁니다.

지금은 **이름·Git 연결**만 확인하면 됩니다. Domains·Env는 건드리지 않아도 됩니다.

실수로 프로젝트를 여러 개 만들어도 연습에는 큰 문제 없습니다.

나중에 Settings에서 Delete할 수 있으니, 지금은 “하나라도 연결됨”에 집중하세요.

화면 참고: *Project 홈 — Deployments 탭 위치*

5-1. 프로젝트 이름만 바뀌도 되나

Import·템플릿 직후 **Project Name**은 Vercel 쪽 표시 이름입니다.

GitHub 저장소 이름과 달라도 되는 경우가 많습니다. 연습용으로 `hello-vercel-lab` 처럼 바뀌도 됩니다.

이름을 나중에 바꾸려면 Project → **Settings** → General 근처의 이름 항목을 보면 됩니다.

초보 한 바퀴에서는 **꼭 바꿀 필요 없습니다.**

중요한 것은 Git 연결이 올바른 저장소를 가리키는지입니다.

실수로 템플릿과 Import를 둘 다 만들어 카드가 두 개면, 연습에 쓸 하나만 골라 Deployments를 보면 됩니다.

쓰지 않는 카드는 나중에 삭제해도 되고, 지금은 무시해도 됩니다.

배포가 이미 Ready여도 4장 순서로 Visit·로그를 다시 보면 개념이 고정됩니다.

이 장에서는 “카드가 생겼다”까지만 체크해도 통과입니다.

6. 이 장에서 안 함

배포 로그 읽기·Visit로 사이트 확인(4장) · Preview vs Production(5장) · 환경변수(6장) · 커스텀 도메인(7장) · CLI · 모노레포 Root 심화

Public에 비밀키·`.env` 파일을 올리지 마세요. 키는 6장 **Environment Variables**로만 둡니다. README에 API 키를 붙여 넣는 실수는 초보에게 흔하니, 푸시 전에 한 번만 더 봅니다.

짧게 정리

GitHub **Connect** → **Add New** → **Project** → **템플릿** 또는 **Import** → Framework 기본값 → (Deploy는 눌러도 됨) → Projects에 카드 확인.

다음 **4장**에서 첫 배포 URL을 열고, Redeploy 기본만 익힙니다.

화면 참고: 4장 예고 — Deployments · Visit

4장. 첫 배포

3장에서 만든 프로젝트의 배포가 끝나는지 보고, URL로 방문한 뒤, **Redeploy**만 익힙니다.

Preview와 Production의 뜻 구분은 5장에서 합니다.

이 장의 성공 기준은 브라우저에서 내 사이트 화면이 한 번이라도 열린 것입니다.

1. 배포란?

배포(Deploy) = 코드를 빌드해서 인터넷 주소로 열리게 하는 일

단계	어디서
코드	GitHub 저장소
빌드·호스팅	Vercel
결과	<code>https://프로젝트이름-xxxx.vercel.app</code> 형태 주소

로컬에서 `npm run build` 를 꼭 성공시켜야만 배포할 수 있는 것은 아닙니다.

Vercel이 클라우드에서 빌드를 대신 돌려 줍니다. (로컬 빌드 연습은 이 책 범위 밖입니다.)

3장에서 Deploy를 이미 눌렀다면, 이 장은 그 결과 확인부터 하면 됩니다.

아직이면 Project에서 **Deploy** / 최신 커밋 배포를 한 번 시작합니다.

화면 참고: Deploy 비유 — GitHub → Vercel 빌드 → URL

2. Deployments 목록 열기

1. Dashboard → 해당 **Project** 클릭
2. **Deployments** 탭
3. 맨 위(최신) 행의 상태 확인

상태에 자주 보이는 말:

상태	의미
Building / Building...	아직 빌드 중 — 잠시 기다림
Ready	성공 — URL로 열 수 있음
Error / Failed	실패 — 로그를 열어 원인 확인
Canceled	취소됨 — 다시 Deploy/Redeploy

Building이 1~3분(템플릿·네트워크에 따라 더) 가는 것은 흔합니다.

새로고침해도 되고, 목록이 자동으로 바뀌는 경우도 있습니다.

같은 행을 연타한다고 더 빨라지지는 않습니다.

커밋 메시지·브랜치 이름이 행에 보이면, “어떤 Git 상태로부터 빌드했는지” 힌트입니다.

지금은 **상태(Ready/Error)** 만 보면 충분합니다.

화면 참고: *Deployments — Building → Ready*

3. Ready면 Visit로 열기

1. Ready인 배포 행 클릭(또는 행의 **Visit** / 링크 아이콘)
2. 새 탭에서 사이트가 보이면 **첫 배포 성공**
3. 주소창 URL을 북마크해 두면 이후 장에서 비교하기 쉽습니다

주소는 보통 아래 형태입니다.

```
https://프로젝트이름.vercel.app
```

또는 배포마다 붙는 더 긴 미리보기 주소입니다.

둘 다 “열렸다”면 이 장 목표는 달성입니다. 어떤 쪽이 Production인지는 5장에서 가릅니다.

템플릿이면 Next.js 기본 랜딩·Vercel 안내 문구가 보여도 정상입니다.

“내가 디자인한 페이지”가 아니어도 **배포 파이프가 통했다**는 것이 핵심입니다.

화면 참고: Visit — 브라우저에 템플릿 첫 화면

여기서 막히면

404·빈 화면이면: 배포가 Ready인지 · 다른 프로젝트 Visit을 연 건 아닌지 · Root Directory가 맞는지(모노레포만)를 봅니다.

인증 벽·로그인 화면만 보이면 템플릿이 원래 그런 것일 수 있습니다. 에러 페이지가 아니면 일단 성공으로 봅니다.

3-1. URL을 공유하기 전에

Visit로 열렸다고 바로 단체 채팅에 붙여 넣기 전에 한 번만 봅니다.

1. 주소가 **본인 프로젝트**인지 (비슷한 이름 프로젝트와 혼동)
2. 화면에 **테스트용 문구·개인정보**가 없는지
3. Private 저장소여도 `*.vercel.app` 주소는 **링크 아는 사람이 열 수** 있는 경우가 많음

연습 단계면 링크를 좁은 범위에만 공유하세요.

비밀번호로 잠그는 보호 기능은 이 책 후반·공식 문서에서 따로 보면 됩니다.

모바일 데이터·회사망에서만 안 열리면 네트워크 문제일 수 있습니다.

다른 기기·시크릿 창으로 한 번 더 Visit해 보세요.

4. 빌드가 Error일 때 (최소)

1. 실패한 배포 행 클릭 → **Building / Logs** (또는 Runtime Logs가 아닌 **Build** 쪽)
2. 빨간 에러 근처 **맨 아래쪽**을 먼저 읽습니다
3. 초보가 자주 고치는 것:
 - GitHub에 **빌드에 필요한 파일**(`package.json` 등)이 푸시됐는지
 - Framework Preset이 완전 엉뚱한 값인지 → Settings → General에서 확인
 - 환경변수 필수 앱인데 아직 비어 있는지 → **6장**에서 넣고 Redeploy
 - 의존성 설치 실패면 네트워크·레지스트리 일시 오류일 수 있어 **Redeploy** 한 번

로그 전체를 이해하려고 하지 마세요. **한 줄 에러 메시지**를 복사해 검색·AI에게 물어보는 정도면 충분합니다.

비밀번호·토큰이 로그에 보였다면 즉시 **그 키를 폐기·재발급**하고, 채팅·이슈·스크린샷에 붙여 넣지 마세요.

화면 참고: *Deployment Logs — Error 한 줄*

5. Redeploy 기본

코드를 안 바꿔도 **다시 빌드**할 때 씁니다. (환경변수 추가 직후, 일시 실패 직후 등)

1. Deployments → 해당 배포 행 ... 메뉴
2. **Redeploy**
3. 옵션이 있으면 초보는 기본값 유지 → 확인
4. 새 행이 Building → Ready 되는지 확인 → 다시 Visit

GitHub에 새 커밋을 푸시하면 Vercel이 자동으로 새 배포를 만드는 경우가 많습니다.

푸시 후 Deployments 맨 위에 새 행이 생기는지 보면 됩니다.

그게 이후 작업의 기본 리듬입니다: **고친다 → 푸시한다 → Ready를 본다 → Visit한다.**

자동 배포가 안 뜨면 Git 연결이 끊겼거나, 해당 브랜치가 무시 설정일 수 있습니다.

3장 Connect를 다시 확인하고, 그래도 안 되면 8장 Q&A로 미뤄도 됩니다.

화면 참고: *... → Redeploy · 푸시 후 새 Deployment 행*

5-1. 배포가 성공했다는 최소 증거

아래 중 **둘만** 만족하면 이 장을 통과한 것으로 보면 됩니다.

1. Deployments에 **Ready** 행이 있다
2. Visit로 브라우저에 **HTML 페이지**(또는 템플릿 첫 화면)가 보인다
3. (선택) README나 페이지를 한 줄 고쳐 푸시하니 **새 행**이 생겼다

콘솔 에러·디자인 깨짐·이미지 깨짐은 “배포 파이프 실패”와는 별개인 경우가 많습니다.
일단 URL이 열린 뒤, 화면 내용은 AI·코드 쪽에서 고치면 됩니다.

팀원에게 “배포해 줘” 요청을 받을 때도 같은 순서로 설명하면 됩니다.

Deployments → Ready → Visit. 로그는 실패할 때만 엽니다.

6. 이 장에서 안 함

Preview vs Production 정의(5장) · 브랜치·PR 배포 전략 · 환경변수 입력(6장) · 커스텀 도메인(7장) · `vercel --prod` CLI · 롤백 심화 · Analytics

Production에 올리는 버튼·체크가 보여도, **의미는 5장에서** 정리한 뒤 쓰는 편이 덜 헷갈립니다.

짧게 정리

Project → **Deployments** → **Ready** 확인 → **Visit**로 URL 열기 → 실패 시 Logs 맨 아래 → 필요하면 **Redeploy** · 이후에는 푸시로 새 배포를 확인.

다음 **5장**에서 Preview(PR/브랜치)와 Production을 구분합니다.

화면 참고: 5장 예고 — Preview / Production 배지

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

5장. Preview vs Production

4장에서 URL을 열어 봤다면, 이번 장은 그 주소가 **Preview**인지 **Production**인지만 분명히 가릅니다.

도메인·환경변수 심화는 다음 장입니다.

혼동하면 “고쳤는데 본편에 왜 안 보이지?”가 바로 나옵니다. 배지·브랜치·URL 형태만 보면 대부분 풀립니다.

1. 한 줄 정의

구분	한 줄
Production	본편. 기본 브랜치(보통 <code>main</code> / <code>master</code>)가 가리키는 배포. 방문자가 쓰는 주소
Preview	미리보기. 다른 브랜치·Pull Request(PR)마다 생기는 배포 주소

Production = “지금 공식으로 열리는 사이트”

Preview = “합치기 전에 먼저 보는 연습·검토용 사이트”

둘 다 Vercel이 빌드해 주지만, **역할이 다릅니다.**

Ready라고 해서 전부 본편은 아닙니다. **Environment 배지**를 보세요.

화면 참고: *Deployments — Production / Preview 배지*

2. 화면에서 구분하는 법

1. Project → **Deployments**
2. 각 행의 **Environment** / 배지 확인
3. 행을 열어 **Branch** 이름과 **URL**을 함께 본다

자주 보이는 표시:

- **Production** — 본편 배포
- **Preview** — 브랜치·PR 배포

Branch	보통
main (또는 Settings의 Production Branch)	Production 후보
feature/... , fix/... , PR 브랜치	Preview

Visit로 연 주소가 프로젝트-git-브랜치-팀.vercel.app 처럼 **길고 브랜치가 섞인 형태면** Preview인 경우가 많습니다.

짧은 프로젝트.vercel.app (또는 연결한 커스텀 도메인)는 Production 쪽에 가깝습니다.

같은 프로젝트 안에 Ready 배포가 여러 개여도 정상입니다.

어느 배지·어느 URL을 보고 있는지 먼저 말하세요.

화면 참고: 배포 상세 — Branch · Environment · URL

3. 왜 나누나 (초보용)

상황	어디에 올리나
문구·UI를 실험 중	Preview (별도 브랜치·PR)
“이제 이게 본편이다”	Production Branch에 합친 뒤 (merge)
실수해도 본편 방문자가 안 보였으면 좋겠다	Preview로 먼저 확인
팀원·나에게 “이 링크 봐줘”	Preview URL 공유

GitHub에서 PR을 열면 Vercel이 댓글·체크에 **Preview URL**을 붙여 주는 경우가 많습니다.

그 링크는 **본편을 바꾸지 않고** 화면만 미리 보는 용도입니다.

PR을 Merge하기 전에는 Production Visit이 예전 화면이어도 대개 정상입니다.

화면 참고: GitHub PR — Vercel Preview 링크 댓글

4. 따라하기 — Preview 하나 만들기

GitHub 기초(브랜치·PR)가 있다면 아래만 하면 됩니다.

없으면 **읽기만** 하고, Deployments에서 Production/Preview 배지만 구분해도 이 장은 통과입니다.

1. GitHub에서 연습 브랜치 생성 (예: `preview-test`)
2. README나 페이지 문구를 **한 줄만** 수정 → 커밋 → 푸시
3. (선택) Pull Request 생성
4. Vercel Project → **Deployments**에 **Preview** 새 행이 생기는지 확인
5. 그 행 **Visit** → 수정한 문구가 보이는지 확인
6. 같은 시각에 Production Visit을 열면 **예전 문구**가 남는 것이 정상인 경우가 많습니다 (아직 merge 전)

```
# 개념만 - 브랜치 이름은 예시
preview-test
```

여기서 막히면

Preview 행이 안 생기면: Git 연결·해당 브랜치 푸시 여부 · Project Settings → Git에서 배포 브랜치를 너무 좁혀 두지 않았는지 확인.

Production만 계속 생기면 Production Branch에 직접 푸시한 것일 수 있습니다. 브랜치 이름을 다시 봅니다.

화면 참고: *Preview Ready* → *Visit* · *Production*은 그대로

5. Production으로 올리는 감각

초보 기본 흐름:

1. Preview에서 화면 확인
2. GitHub에서 PR **Merge** (또는 연습 저장소에서만 Production Branch에 직접 푸시)

3. Vercel Deployments에 **Production** 새 행(또는 갱신)

4. Production URL Visit → 변경 반영 확인

Dashboard의 **Promote to Production** 같은 버튼이 보이면, Preview 배포를 본편으로 승격하는 기능입니다.

의미만 알고, 회사·공유 Production에서는 함부로 누르지 마세요. **연습 프로젝트에서만** 시도합니다.

Settings → **Git** → Production Branch가 `main` (또는 본인이 쓰는 기본 브랜치)인지 한 번만 눈으로 확인하면 충분합니다.

이름을 지금 바꿀 필요는 없습니다. “본편이 어느 줄기를 따르는지”만 알면 됩니다.

캐시·CDN 때문에 반영이 수십 초 늦을 수 있습니다.

안 보이면 시크릿 창·강제 새로고침 후, Deployments의 **생성 시각**이 방금인지 확인하세요.

화면 참고: Settings → Git — Production Branch = main

5-1. 자주 하는 착각

착각	실제
Ready면 무조건 본편	Preview Ready 일 수 있음
Preview에서 고치면 Production도 자동 변경	Merge(또는 Production 브랜치 푸시) 전까지 본편은 그대로인 경우가 많음
URL이 열리니 배포 종류는 상관없다	공유·북마크 전에 배지 를 확인
PR을 닫으면 Preview가 영원히 남는다	보통 더 이상 안 쓰게 되거나 만료·정리 대상 — 본편 대체 아님

연습할 때 탭을 두 개 열어 두면 헷갈림이 줄어듭니다.

한쪽은 Production Visit, 다른 쪽은 Preview Visit. 문구가 다르면 구분에 성공한 것입니다.

브랜치 이름에 한글·공백을 넣으면 URL·도구가 불편해질 수 있습니다.

연습 브랜치는 `preview-test` 처럼 **영문·하이픈**이면 충분합니다.

5-2. 한 문장으로 말해 보기

다음 문장을 그대로 읽어 보세요. 동의하면 이 장 개념은 충분합니다.

- “**Preview**는 브랜치·PR용 미리보기 주소이고, **Production**은 기본 브랜치 본편이다.”
- “Preview에서 문구가 보여도 Merge 전이면 Production은 예전일 수 있다.”
- “공유 전에 Deployments **배지**와 Branch를 확인한다.”

GitHub 초간단에서 PR까지 했다면, Vercel Preview 링크는 PR 검토의 **화면 증거**로 쓰면 됩니다. 코드 탭만 보지 말고 Visit 한 번이 빠릅니다.

프로덕션 도메인·HTTPS 인증서는 연결만 되면 Vercel이 처리하는 경우가 많습니다. 그 “붙이기” 자체는 7장 맛보기에서 버튼만 따라가면 됩니다.

6. 이 장에서 안 함

환경변수 Preview/Production 분리 입력(6장) · 커스텀 도메인(7장) · 보호된 Production·비밀번호 게이트 심화 · 트래픽·Analytics · 롤백 UI 전부

“본편 주소에 내 도메인을 붙이기”는 7장 맛보기입니다.

API 키를 Preview에만 넣고 Production에 안 넣는 실수는 6장에서 다룹니다.

Production URL을 북마크할 때는 Deployments에서 **Production** 배지가 달린 행의 Visit을 쓰세요.

Preview 긴 주소를 본편인 줄 알고 공유하면, 브랜치를 지운 뒤 링크가 애매해질 수 있습니다.

짧게 정리

Production = 기본 브랜치 본편 · **Preview** = 브랜치/PR 미리보기 · Deployments 배지 · Branch·URL로 구분 · 실험은 Preview → 확인 후 Merge.

다음 6장에서 API 키 등을 **Environment Variables**로 넣는 법을 합니다.

화면 참고: 6장 예고 — Settings → Environment Variables

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

6장. 환경변수(.env)

5장까지 **Preview / Production**을 봤습니다.

이 장은 **비밀값**을 코드 밖에 두는 법입니다. API 키·비밀번호를 저장소에 넣지 않습니다.

학습 목표

- Vercel 대시보드에 환경변수 추가
- 로컬은 `.env.local` · 커밋하지 않음
- Production / Preview(·Development) 구분을 말로

1. 환경변수란?

환경변수 = 코드에 직접 쓰지 않는 설정·비밀값

예: API 키, DB 주소, `NEXT_PUBLIC_*` 같은 공개 설정

어디에	파일·화면	커밋?
로컬 PC	<code>.env.local</code>	✗
Vercel	Project → Settings → Environment Variables	(대시보드만)
저장소	<code>.env</code> / 키 문자열	✗ 비밀은 금지

화면 참고: 코드에 키 넣기 ✗ · 환경변수 ✔

2. 시작 전

- 4~5장에서 배포한 본인 프로젝트 하나

- vercel.com 로그인 · 해당 Project 열림
- 연습용 키면 **가짜 값**으로 충분 (실제 유료 키는 나중에)

실제 키가 있으면 이 장 연습 후 **Rotate(재발급)** 습관을 들이세요. 이 책은 재발급 UI를 깊게 다루지 않습니다.

3. Vercel 대시보드에 넣기

1. vercel.com → 해당 **Project**
2. **Settings** → **Environment Variables**
3. **Key / Value** 입력 (예: Key `DEMO_API_KEY` , Value `practice-only-not-real`)
4. 적용 환경 체크: **Production** · **Preview** · (있으면) **Development**
5. **Save**

화면 참고: *Settings* → *Environment Variables* → *Key / Value* / 환경 체크

여기서 막히면

메뉴 이름이 조금 달라도 **Settings** 안의 **Environment Variables**를 찾습니다.
저장 후 값이 안 보이면 목록에 Key만 보이고 Value는 가려진 것이 정상인 경우가 많습니다.

4. Production vs Preview

같은 Key라도 **환경마다 다른 Value**를 둘 수 있습니다.

환경	언제 쓰이나	이 책에서
Production	main(등) 프로덕션 배포	실제/최종용 값
Preview	PR·브랜치 미리보기	연습·테스트용 값
Development	<code>vercel env</code> / 로컬 연동(심화)	이름만 · 필수는 <code>.env.local</code>

초보는 **둘 다 같은 연습 값**으로 시작해도 됩니다.

나중에 “미리보기는 테스트 키, 프로덕션은 진짜 키”로 나누면 됩니다.

화면 참고: *Production / Preview 체크박스*

값을 바꾼 뒤에는 **새 배포**가 필요할 수 있습니다. Deployments에서 Redeploy(또는 푸시 한 번)로 반영을 확인합니다.

5. 로컬 `.env.local`

1. 프로젝트 루트에 `.env.local` 파일 생성
2. 한 줄 예: `DEMO_API_KEY=practice-only-not-real`
3. 프레임워크가 읽는 이름인지 확인 (Next.js 등 · 문서의 `NEXT_PUBLIC_` 규칙은 공식 Docs)
4. **Git에 올리지 않음**

화면 참고: *에디터에 .env.local · 탐색기에 점 파일*

`.gitignore`에 보통 `.env*.local` / `.env`가 있습니다. 없으면 추가한 뒤 커밋합니다(파일 이름만 무시 규칙 · 비밀값 본문은 커밋하지 않음).

6. 절대 하지 말 것

- README·이슈·채팅에 실제 API 키 붙여 넣기
- `.env` / `.env.local`을 커밋·푸시
- 공개 저장소에 `SECRET=` 문자열
- 스크린샷에 Value 전체 노출

유출 의심 시: 해당 서비스에서 **키 폐기·재발급** → Vercel·로컬 값 교체 → 재배포.

“어떻게 해킹했는지” 같은 우회는 이 책 범위 밖입니다.

화면 참고: *.gitignore에 .env.local · git status에 안 잡힘*

미션

1. 대시보드에 Key 하나 추가 (가짜 Value) · Production + Preview 체크 · Save
 2. 로컬에 `.env.local` 같은 Key 한 줄 · **커밋되지 않음** 확인
 3. (선택) Redeploy 후 앱에서 값이 읽히는지 — 프레임워크별로 다름, 실패해도 8장
-

이 장에서 안 함

Edge Config · 암호화 KMS 심화 · `vercel env pull` 필수화 · 결제·OAuth 전체 설계 · 모노레포 env 공유

짧게 정리

비밀은 **대시보드 + `.env.local`** · 저장소에 넣지 않음 · Production/Preview를 나눌 수 있음 · 값 변경 후 재배포를 의심 · **도메인 맛보기는 7장.**

화면 참고: 7장 예고 — Domains

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

7장. 커스텀 도메인 맛보기

6장까지 배포·환경변수를 봤습니다.

이 장은 **내 도메인을 붙이는 맛보기**만입니다. DNS 심화·구매·이전이 목표가 아닙니다.

학습 목표

- Project에 도메인 추가 화면까지
 - DNS는 **고수준 힌트만**
 - 무료 `*.vercel.app` 은 항상 쓸 수 있음
-

1. 왜 맛보기만?

커스텀 도메인 = `my-site.com` 처럼 **내가 가진 이름**을 Vercel 프로젝트에 연결하는 것.

✅ 이 장	Add Domain 화면 · DNS에 “어디로 가리키라” 힌트 · <code>*.vercel.app</code> 유지
❌ 이 장	도메인 구매 대행 · 레지스트라별 클릭 전부 · 이메일 DNS·CDN 심화

도메인이 없어도 **배포 연습은 이미 가능합니다.**

기본 주소 `프로젝트명.vercel.app` 은 무료 플랜에서도 계속 열립니다.

화면 참고: 기본 `.vercel.app` URL · Domains 메뉴*

2. 시작 전

- 4장에서 배포 성공한 Project
- (선택) 이미 구매한 도메인 하나 — **없으면 이 장 읽기만** 하고 넘어가도 됨

- DNS 변경 권한(도메인 관리 사이트 로그인)

도메인이 없으면: **섹션 5(*.vercel.app)**만 확인하면 이 장 통과입니다.

3. Vercel에 도메인 추가

1. Project → **Settings** → **Domains** (또는 상단 Domains)
2. 입력란에 `example.com` 또는 `www.example.com`
3. **Add**
4. 화면에 나오는 **DNS 안내**(A / CNAME 등)를 메모

화면 참고: *Domains* → *Add* → *DNS 지시 문구*

문구·추천 레코드는 시점에 따라 바뀝니다. **화면에 적힌 그대로** 도메인 관리 사이트에 맞춥니다.

여기서 막히면

Invalid / already used → 오타·다른 Vercel 계정에 붙어 있는지.

권한 오류 → 그 도메인을 **소유한 계정**인지.

심화 이전·해제 절차는 공식 Docs · 이 책은 “추가 화면까지”.

4. DNS 팁 (고수준만)

도메인 관리 사이트(가비아·Cloudflare·Namecheap 등)에서 **레코드**를 고칩니다.

흔한 경우	느낌
apex(<code>example.com</code>)	A 레코드 등으로 Vercel이 안내한 대상
<code>www</code>	CNAME → Vercel이 안내한 호스트
반영 시간	수 분~수십 분(길면 더) · 새로그침·시크릿 창

화면 참고: *DNS 레코드 표* — *타입 / 이름 / 값* (예시 가림)

- 예전에 쓰던 A/CNAME과 충돌하면 안내대로 정리
- HTTPS 인증서(SSL)는 Vercel이 알아서 발급하는 경우가 많음 · 발급 전이면 Validating 상태일 수 있음
- 레코드 값을 채팅·스크린샷에 무분별 공유하지 않기(계정·존 구조 노출)

이 책은 레지스트라별 화면을 하나씩 찍지 않습니다. **Vercel이 준 지시 = 도메인 사이트에 동일 입력.**

5. 무료 *.vercel.app 은 항상

커스텀 도메인이 실패해도, 아직 없어도:

- `https://내프로젝트.vercel.app` (또는 배포마다 붙는 Preview URL)
- Production 배포가 살아 있으면 그 주소로 접속 가능

연습·포트폴리오 초안·나두 실습은 ***.vercel.app** 만으로 충분한 경우가 많습니다.

커스텀은 “필요할 때” 다시 Settings → Domains.

화면 참고: *Deployments*의 `.vercel.app` 링크 Visit*

미션

도메인 있음: Add Domain → DNS 안내에 맞춰 1개 레코드 → Valid / 접속 확인(시간 여유) · 실패해도 `*.vercel.app` 으로 동작 확인

도메인 없음: Domains 화면만 열어보기 → Production URL(`*.vercel.app`) Visit → **8장으로**

이 장에서 안 함

도메인 구매·이전·만료 · 이메일(MX) · 멀티존·와일드카드 심화 · 기업 SSO · 유료 플랜 비교 전부

짧게 정리

맞보기 = **Add Domain + DNS 안내에 맞추기** · 반영은 기다림 · 막혀도 ***.vercel.app** 은 그대로 · 자주 막히는 장면은 8장.

화면 참고: 8장 예고 — Q&A

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

8장. 자주 막히는 장면 Q&A

새 기능을 배우지 않습니다. 2~7장에서 막힐 때 **먼저 볼 곳**만 짚습니다.

본인 계정·본인 GitHub·본인 Project 기준. 무단 접근·우회는 다루지 않습니다.

쓰는 법: 가까운 Q → “먼저 볼 곳” 위부터 → 안 되면 복구 순서 → 범위 밖 표.

화면 참고: 8장 지도 — 빌드 / GitHub / 브랜치 / env / 404 / 범위 밖

A. 빌드 실패 (Build failed)

Q1. Deployments에 Error / Failed가 뜹니다

1. 해당 Deployment 클릭 → **Building / Logs**
2. 빨간 줄·Error: 근처를 위에서부터
3. 로컬에서 같은 명령이 되는지(예: `npm run build`) — 되면 Vercel 설정·환경변수 의심
4. Root Directory / Framework Preset이 프로젝트와 맞는지(3~4장)

로그 전체를 남에게 붙여 넣기 전에 **키·토큰·이메일**을 가리세요.

Q2. 의존성·모듈을 못 찾는다고 합니다

`package.json` 이 커밋됐는지 · lock 파일 · Node 버전(Project Settings → General).
모노레포·커스텀 빌드 심화는 범위 밖 → 공식 Docs.

화면 참고: Deployment → Build logs 일부(개인정보 가림)

B. GitHub이 연결되지 않음

Q3. Import 목록에 저장소가 없습니다

1. vercel.com · GitHub **같은 사람**인지
2. GitHub → Settings → Applications → Vercel **권한**에 그 repo/org가 있는지

3. Adjust permissions / Grant로 저장소 허용 후 Vercel 새로고침
4. Private이면 권한이 더 필요할 수 있음

Q4. “Connect Git Repository”에서 멈춥니다

팝업 차단 · 다른 브라우저 · Sign out 후 GitHub로 다시 Sign in.
조직(org) 저장소면 org 관리자 승인 대기일 수 있음.

화면 참고: *GitHub Applications — Vercel Repository access*

C. 프로덕션이 엉뚱한 브랜치

Q5. Preview는 맞는데 Production이 옛날입니다

1. Project → **Settings** → **Git** (또는 Production Branch)
2. Production Branch가 `main` (또는 팀이 쓰는 이름)인지
3. 최신 커밋이 그 브랜치에 **푸시**됐는지 (깃허브 책 6장)
4. Deployments에서 Production 행의 커밋 해시·메시지 확인

Q6. PR Preview만 갱신됩니다

정상일 수 있음. Production은 **프로덕션 브랜치 머지·푸시** 후 갱신.
5장 Preview vs Production을 다시 보면 됩니다.

화면 참고: *Settings → Production Branch = main*

D. 환경변수가 없음 / 안 먹음

Q7. 로컬은 되는데 Vercel만 실패합니다

1. Settings → Environment Variables에 Key 철자
2. **Production / Preview** 중 배포 종류에 체크됐는지 (6장)
3. 값 저장 후 **Redeploy** 했는지 (옛 배포는 옛 env)
4. `NEXT_PUBLIC_` 등 **빌드 시 박히는 이름**인지 — 런타임만 읽는 키와 다름(프레임워크 Docs)

Q8. `.env.local` 을 푸시했습니다

1. 즉시 키 **폐기·재발급**
2. Git에서 시크릿 제거(히스토리 심화는 밖 — 최소한 다음 커밋에서 삭제·gitignore)
3. Vercel 값은 대시보드로만
4. 공개 로그·이슈에 붙여 넣은 적 있는지 확인

화면 참고: *Environment Variables* 목록 · *Redeploy*

E. 404 · 안 열림

Q9. Visit 했더니 404

1. URL 오타 · 배포가 Ready인지
2. 프레임워크 **output** / **basePath** · 정적 export 경로
3. 커스텀 도메인이면 DNS·SSL이 Validating은 아닌지 (7장) · 우선 `*.vercel.app` 으로 같은 배포 확인
4. 로컬 `npm run build` + 미리보기와 경로 비교

Q10. 예전에 되던 주소가 忽然 404

다른 Project / 팀 · 프로젝트 이름 변경 · 배포 삭제·롤백 · 도메인이 다른 프로젝트로 옮김.
Deployments 목록에서 **현재 Production** Visit 링크를 다시 복사.

F. 영문 · 범위 밖

비슷한 문구	뜻
Deployment / Ready / Error	배포 / 성공 / 실패
Build logs	빌드 기록
Production / Preview	최종본 / 미리보기

비슷한 문구	뜻
Environment Variables	환경변수
Domains / Validating	도메인 / 검증 중
Production Branch	프로덕션에 쓰는 브랜치

주제	이 책	다음
Actions 심화	✗	GitHub + Vercel Docs
Edge 미들웨어	✗	공식 Docs
모노레포·Turborepo	✗	나중에
결제·쇼핑몰	✗	범위 밖
DNS·이메일 심화	✗	7장은 맛보기만

복구 순서 (최소)

빌드: Logs → 로컬 build → Root/Framework → env

GitHub: 같은 계정 → Vercel app 권한 → 새로그침

브랜치: Production Branch · 그 브랜치에 푸시 · Deployments 커밋

env: Key 철자 · 환경 체크 · Redeploy · 시크릿 유출 시 재발급

404: Ready URL · *.vercel.app 먼저 · 도메인이면 DNS

증상별 복습: 계정 2 · Git 연결·프로젝트 3 · 첫 배포 4 · Preview/Prod 5 · env 6 · 도메인 7 ·
막힘 8 · 미션 9

짧게 정리

막히면 새 클릭보다 **Logs · 권한 · 브랜치 · env · URL** 순서 · 시크릿은 재발급 · 도메인 전에도
*.vercel.app 으로 확인 · 다음은 **9장** 미니 미션.

화면 참고: 9장 예고 — 미니 미션

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

9장. 다음에 연습할 것

본편 한 바퀴를 끝냈습니다. 새 기능을 깊게 배우지 않습니다.

혼자 돌릴 **미니 미션**, **한 바퀴 더** 점검, “나중에 볼 주제” 이름만, 그리고 **부록 두 개** 메뉴입니다.

연습 프로젝트·README에 실명·실제 이메일·API 키를 넣지 마세요. 이름은 `practice-vercel-01` 처럼 평범하게.

학습 목표

미션 3~5개 고르기 · 계정→…→env/도메인 복습 · 나중에 볼 주제는 이름만 · 부록 A·B를 다음 메뉴로

1. 미니 미션 (오늘 3개면 충분)

A. 다시 배포 — README 한 줄 → 푸시 → Deployments Ready → Visit (4장)

B. Preview 한 번 — 짧은 브랜치·PR → Preview URL 확인 · Production과 다름 (5장) · 타인 repo PR 금지

C. env 연습 — 대시보드에 가짜 Key · `.env.local` · 커밋 안 됨 · (선택) Redeploy (6장)

D. 도메인 맛보기 — Domains 화면 열기 · 없으면 `*.vercel.app` 만 Visit (7장)

E. “왜 안 되지?” — 일부러 Logs 한 번 읽어 보기 · 막히면 8장 순서

화면 참고: 미션 A~E — 오늘 고른 3개

막히면 8장 증상별 표로. 새 버튼보다 먼저 볼 곳.

2. 한 바퀴 더

☐ 계정 Vercel 로그인 (2)

- ☐ **GitHub 연결** · Project 하나 (3)
- ☐ **첫 배포** Ready · Visit (4)
- ☐ **Preview vs Production** 구분 (5)
- ☐ **환경변수** 대시보드 + `.env.local` · 비밀 미커밋 (6)
- ☐ **도메인** 맛보기 또는 `*.vercel.app` (7)
- ☐ 막히면 8장 순서

계정 → GitHub 연결 → 배포 → Preview/Prod → env → (맛보기) 도메인

화면 참고: 한 바퀴 여섯 칸

3. 나중에 — 이름만

주제	한 줄	이 책
<code>vercel env pull</code>	대시보드 env를 로컬로	6장 최소
롤백	이전 Production으로 되돌리기	Deployments UI 이름만
Analytics / Speed Insights	방문·성능 숫자	나중에
Edge 미들웨어	요청 중간에 로직	나중에
모노레포	한 저장소에 앱 여러 개	나중에

팀 규칙·유료 플랜 비교는 이 장에서 하지 않습니다.

4. 일부러 나중에

GitHub Actions 심화 · Edge · 대규모 모노레포 · 결제·쇼핑몰 — **한 바퀴(계정배포env)** 가 먼저.
이 책 성공 기준은 쇼핑몰이 아닙니다.

5. 다음 메뉴 — 부록

부록 A 부록-나두따라하기실습.md — 나두 영상 자료 → Vercel 배포 한 바퀴 (Lab).

부록 B 부록-템플릿배포실습.md — 가벼운 템플릿 Import → Deploy.

고르는 법: 영상·나두 흐름이 익숙하면 A 하나 · 템플릿으로 손맛만 보면 B 하나 · 둘 다면 A 후 B(하루에 하나만도 OK).

시크릿·실명 규칙은 부록 안내를 따릅니다.

화면 참고: 부록 A / B 표지

본편 마무리

1. 혼자 반복 · 미션 3개면 충분
2. 복습: 계정→연결→배포→Preview/Prod→env→도메인 맛보기
3. Analytics·Edge·모노레포는 미룸
4. 다음은 부록 실습 · 막히면 8장

고맙습니다. 본편 한 바퀴를 끝까지 따라와 주셔서.

화면 참고: 본편 완주 — 한 바퀴 + 부록 A/B

나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』

부록 A. 나두식 따라하기 Lab — Vercel에 올려 보기

순수 설치·클릭·배포. 나두Ai 영상에서 본 **AI 워크플로 결과물**을 Vercel에 **한 번** 올리는 실습장.

본편 1~6장 = 계정 · GitHub 연결 · 첫 배포 · Preview/Production · 환경변수 참고.

이 부록 = **영상 재료** → **배포 URL**. 무료 이북.

영상 제목은 자리표시 **나두 영상에서 본 ○○ — 본인이 고른 영상**으로 바꿔 읽으면 됩니다.
UI 문구가 조금 달라도 **비슷한 버튼**이면 됩니다. 주소는 **시청한 영상·공식 안내**만.

쓰는 법

1. Lab **하나** 고르기 → 해당(또는 비슷한) 나두 영상 틀기 → 재료 **1개**.
2. **따라하기** 번호대로: 만들기 → GitHub(필요 시) → Vercel Deploy → URL 열기.
3. **끝**에서 배포 URL만 확인하면 완료.

시크릿 — 인트로에서 한 번만

해도 됨	하지 말 것
Vercel Environment Variables 에 키 입력	<code>.env</code> / <code>sk-실제키</code> 커밋·푸시
<code>.env.example</code> 에 키 이름만	README·캡처에 키 전체
<code>.gitignore</code> 에 <code>.env</code>	“나중에 지우기”로 방치
실수 시 즉시 키 폐기·재발급	공개 저장소에 실키 남기기

공통 — Vercel에서 URL 확인

1. <https://vercel.com> 로그인.

2. **Dashboard** → 해당 프로젝트.
3. **Visit** / 도메인 링크(예: *.vercel.app) 클릭.
4. 브라우저에 페이지(또는 JSON)가 보이면 성공.

Lab 끝에서는 「**배포 URL 확인**」 한 줄만 보면 됩니다.

실습 목차

Lab	자리표시 영상	따라 할 일	끝
1	나두 영상에서 본 랜딩·소개 페이지	정적 HTML 1장 → Vercel	배포 URL
2	나두 영상에서 본 Next·스타터	Next.js 템플릿 Deploy	배포 URL
3	나두 영상에서 본 카피·문구 AI	문구 1줄 수정 → 재배포	Preview/Prod URL
4	나두 영상에서 본 API·함수 맛보기	API Route hello → 호출	/api/... URL
5	나두 영상에서 본 API 키·연동	Env에 자리표시 키 → 화면만	Env 저장 + URL
6	나두 영상에서 본 포트폴리오·링크모음	정적/템플릿 1개 Deploy	배포 URL
7	나두 영상에서 본 GitHub 공개 예제	Import Git Repo → Deploy	배포 URL

7개 전부 필수는 아님. **하나라도** Deploy → URL이면 충분.

Lab 1. 정적 랜딩 한 장 배포

목표: AI로 만든(또는 손으로 쓴) `index.html` 1장을 Vercel에 올려 `*.vercel.app` 로 연다.

따라하기

1. 메모장·에디터에 `index.html` 새로 만들기.
2. 아래를 붙여 넣고, 제목 · 한 줄 소개 만 본인 문구로 바꾸기.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>제목</title>
</head>
<body>
  <h1>제목</h1>
  <p>한 줄 소개</p>
</body>
</html>
```

3. 폴더 예: `nadoo-lab01` — 그 안에 `index.html` 만 두기.
4. `https://github.com` → + → **New repository** → name `nadoo-lab01` → **Public** → **Create repository**.
5. **Add file** → **Upload files** → `index.html` 끌어다 놓기 → **Commit changes**.
6. `https://vercel.com/new` → **Import**에서 방금 저장소 선택(GitHub 연결은 본편 3장).
7. Framework Preset이 **Other** / 비어 있으면 그대로 → **Deploy**.
8. 완료 후 **Visit** → 제목·한 줄 소개가 보이면 성공.

끝: 배포 URL 확인 · (선택) 메모에 `lab01` + URL.

Lab 2. Next.js 스타터 원클릭 배포

목표: 공식 Next.js 템플릿을 Deploy 한 번으로 올리고 URL을 연다.

따라하기

1. <https://vercel.com/templates/nextjs> 열기.
2. **Next.js Boilerplate**(또는 목록의 가벼운 Next 스타터 1개) 카드 열기.
3. **Deploy / Deploy to Vercel** 클릭.
4. GitHub 저장소 이름 확인(기본값 OK) → **Create / Deploy**.
5. 빌드 로그가 초록 **Ready**가 될 때까지 대기.
6. **Visit** → Next 기본 화면이 보이면 성공.

템플릿 이름이 UI에서 조금 달라도 **Next.js + Deploy 버튼**이면 됩니다.

끝: 배포 URL 확인 · lab02 + URL.

Lab 3. AI 카피 1줄 수정 후 재배포

목표: Lab 1 또는 Lab 2 프로젝트에서 문구 **1줄**만 바꾸고, Preview 또는 Production URL을 다시 연다.

따라하기

1. 나두 영상·AI에 랜딩 히어로 문장 1개, 한국어, 20자 이내 요청 → 결과 1줄 복사.
2. GitHub에서 해당 저장소 → `index.html` 또는 `app/page.tsx` / `pages/index.tsx` 등 **문구가 있는 파일** 열기.
3. **Edit** (연필) → 히어로/제목 문구만 새 문장으로 교체 → **Commit changes**.
4. Vercel Dashboard → 프로젝트 → **Deployments**.
5. 새 Deployment가 **Building** → **Ready** 될 때까지 대기.
6. 해당 행 **Visit** (또는 Production 도메인) 클릭.
7. 바꾼 문구가 보이면 성공.

끝: Preview 또는 Production URL 확인 · `lab03` + URL.

Lab 4. 간단한 API Route 노트

목표: Next 프로젝트에 hello JSON API 한 줄을 두고, 배포 URL의 `/api/...` 를 브라우저로 연다.

따라하기

1. Lab 2에서 만든 Next 저장소(없으면 Lab 2 먼저)를 GitHub에서 연다.
2. App Router면 `app/api/hello/route.ts` **Create new file.** (Pages Router면 `pages/api/hello.ts`)
3. App Router 예:

```
export function GET() {  
  return Response.json({ ok: true, message: "hello from nadoo lab04" });  
}
```

Pages Router 예:

```
export default function handler(req, res) {  
  res.status(200).json({ ok: true, message: "hello from nadoo lab04" });  
}
```

4. **Commit changes.**
5. Vercel Deployment **Ready** 대기.
6. 브라우저 주소창: `https://내프로젝트.vercel.app/api/hello` 입력 → Enter.
7. `{"ok":true,...}` 형태가 보이면 성공.

끝: `/api/hello` URL 확인 · `lab04` + URL.

Lab 5. 환경변수에 API 키 데모 (커밋 없이)

목표: API 키를 **Vercel Environment Variables**에만 넣고, 저장소에는 이름만 남긴다. 실키는 커밋하지 않는다.

따라하기

1. 연습용 Next 저장소(Lab 2) 열기.
2. 루트에 `.env.example` 파일 추가(커밋 OK):

```
DEMO_API_KEY=
```

3. `.gitignore`에 `.env` / `.env.local`이 있는지 확인. 없으면 한 줄 추가 후 커밋.
4. Vercel Dashboard → 프로젝트 → **Settings** → **Environment Variables**.
5. Key: `DEMO_API_KEY` · Value: `demo-not-a-real-key` (연습 문자열) → Environment는 Production/Preview 체크 → **Save**.
6. (선택) 코드에서 `process.env.DEMO_API_KEY`를 **화면에 출력하지 않는** 확인용만 `—` 값이 보이면 안 됨.
7. **Deployments** → 최신 배포 옆 메뉴 → **Redeploy**(환경변수 반영).
8. **Visit** → 사이트는 열리고, GitHub 파일 목록에 실키가 **없으면** 성공.

실제 OpenAI 등 키를 쓸 때도 동일: **Vercel Env만** · `.env.example`은 이름만 · `.env`는 푸시 금지.

끝: Env 저장됨 + 배포 URL 확인 · 시크릿 미커밋 · `lab05`.

Lab 6. 포트폴리오·링크모음 한 장

목표: 나두 영상에서 본 “소개·링크 모음”을 `index.html` 한 장으로 만들어 Vercel에 올린다.

따라하기

1. `index.html` 에 이름 1줄 · 링크 3개(`<a href=“...”>`)만 넣기(실명·직장 메일 대신 닉네임 OK).
2. GitHub 새 저장소 `nadoo-lab06` → `index.html` Upload → Commit.
3. <https://vercel.com/new> → 해당 저장소 **Import** → **Deploy**.
4. **Ready** → **Visit**.
5. 이름·링크 3개가 보이면 성공.

끝: 배포 URL 확인 · `lab06` + URL.

Lab 7. GitHub 공개 hello 저장소 Import

목표: 남이 공개한 가벼운 hello/예제 저장소를 Vercel **Import**로 배포한다.

따라하기

1. 공개 예제 하나 고르기(공식·허용된 것만). 예:
 - <https://github.com/vercel/next.js/tree/canary/examples/hello-world>
 - 또는 나두 영상에서 안내한 **공개** hello 저장소 URL
2. <https://vercel.com/new> 열기.
3. **Import Third-Party Git Repository** / 저장소 URL 붙여 넣기(또는 GitHub 검색 후 Import).
4. 프레임워크 자동 감지 확인 → **Deploy**.
5. **Ready** → **Visit**.
6. 예제 기본 화면이 보이면 성공.

private·유료·키 필수 예제는 이 Lab에서 건너뛰고, **키 없이 뜨는** 공개 예제만.

끝: 배포 URL 확인 · `lab07` + URL.

마치며

- 막히면 본편 **2~6장**(계정 · GitHub · 배포 · Preview · Env).
- 다음: **부록 B** (공식 템플릿만 가벼운 클릭 배포).
- 영상 제목은 자리표시 — 같은 주제면 Lab 번호만 맞추면 됨.

나두Ai · 부록 A · 따라 하기 · 무료 이북

부록 B. 가벼운 템플릿 배포 Lab

공식 Vercel Templates · HTML · (선택) Astro · GitHub Import만.

클릭 → Deploy → URL. 나두Ai · 『AI 하는 사람을 위한 Vercel 초간단』 · 무료 이북.

전제: 본편 2~4장(계정 · GitHub 연결 · 첫 배포).

쓰는 법

1. 목표 읽기
2. 공식 URL 열기
3. 번호 클릭대로 Deploy
4. Visit로 URL 확인

- 버튼 문구가 조금 달라도 **Deploy / Create / Visit**이면 됩니다.
- 무거운 DB·Auth·결제 템플릿은 이 부록에서 **제외**.

시크릿 — 인트로에서 한 번만

올리면 안 됨	올려도 됨
<code>.env</code> 실값, API 키	템플릿 기본 코드
결제·사내 토큰	배포 URL 메모

이 부록 Lab은 **키 없이** 끝나는 템플릿만 고름.

실습 목차

Lab	무엇을	공식 출발점
1	Next.js Boilerplate	vercel.com/templates

Lab	무엇을	공식 출발점
2	HTML Starter	vercel.com/templates
3	Astro Boilerplate	vercel.com/templates
4	Portfolio Starter Kit	vercel.com/templates
5	GitHub 공개 hello Import	vercel.com/new
6	SvelteKit Boilerplate	vercel.com/templates

6개 전부 불필요 — **하나라도** Deploy → Visit이면 완료.

Lab 1. Next.js Boilerplate

목표: 공식 Next.js 보일러플레이트를 원클릭 배포하고 URL을 연다.

공식 URL: <https://vercel.com/templates/nextjs>

따라하기

1. 위 URL 열기(또는 <https://vercel.com/templates> → Next.js 필터).
2. **Next.js Boilerplate** 카드 클릭.
3. **Deploy / Deploy to Vercel** 클릭.
4. Vercel 로그인 상태 확인(본편 2장).
5. GitHub에 복제될 저장소 이름 확인 → **Create / Deploy**.
6. 로그가 **Ready**가 될 때까지 대기.
7. **Visit** 클릭.
8. Next 기본 시작 화면이 보이면 성공.

끝: 배포 URL 확인.

Lab 2. HTML Starter (정적)

목표: 공식 HTML Starter를 배포하고 정적 페이지 URL을 연다.

공식 URL: <https://vercel.com/templates> (검색창에 `HTML Starter`)

따라하기

1. <https://vercel.com/templates> 열기.
2. 검색: `HTML` 또는 `HTML Starter`.
3. **HTML Starter** 카드 열기.
4. **Deploy** 클릭.
5. 저장소 이름 확인 → **Deploy**.
6. **Ready** → **Visit**.
7. HTML 기본 페이지가 보이면 성공.

목록에 이름이 조금 다르면 **순수 HTML · Other 프레임워크**로 표시된 가벼운 스타터 1개로 동일 클릭.

끝: 배포 URL 확인.

Lab 3. Astro Boilerplate

목표: 원클릭 Astro 템플릿을 배포하고 URL을 연다.

공식 URL: <https://vercel.com/templates/astro/astro-boilerplate>

따라하기

1. 위 URL 열기(또는 Templates에서 Astro Boilerplate 검색).
2. **Deploy** 클릭.
3. GitHub 저장소 생성 확인 → **Deploy**.
4. **Ready** → **Visit**.
5. Astro 예제 첫 화면이 보이면 성공.

끝: 배포 URL 확인.

Lab 4. Portfolio Starter Kit

목표: 공식 포트폴리오 스타터를 배포하고 URL을 연다.

공식 URL: <https://vercel.com/templates> (검색: Portfolio Starter Kit)

따라하기

1. Templates에서 **Portfolio Starter Kit** 카드 열기.
2. **Deploy** 클릭.
3. 저장소 이름 확인 → **Deploy**.
4. **Ready** → **Visit**.
5. 포트폴리오 기본 레이아웃이 보이면 성공.

끝: 배포 URL 확인.

Lab 5. GitHub 공개 hello Import

목표: 공개 GitHub 예제를 Vercel Import로 배포한다(템플릿 버튼 없이 URL로).

공식 출발: <https://vercel.com/new>

예제 저장소(교체 가능): <https://github.com/vercel/next.js/tree/canary/examples/hello-world>

따라하기

1. <https://vercel.com/new> 열기.
2. **Import Git Repository** — GitHub 연결돼 있으면 목록에서 고르거나,
3. 공개 URL을 쓸 때: Third-Party / 저장소 URL 칸에
`https://github.com/vercel/next.js` 입력 후, Root Directory를 `examples/hello-world` 로 지정(화면 안내대로).
4. Framework Preset이 Next.js로 잡히는지 확인 → **Deploy**.
5. Root Directory 지정이 어렵면: Templates의 **Next.js Boilerplate**(Lab 1)로 대체해도 이 Lab 목표는 달성.
6. **Ready** → **Visit**.
7. hello 기본 화면이 보이면 성공.

끝: 배포 URL 확인.

Lab 6. SvelteKit Boilerplate

목표: 공식 SvelteKit 보일러플레이트를 원클릭 배포하고 URL을 연다.

공식 URL: <https://vercel.com/templates> (검색: SvelteKit Boilerplate)

따라하기

1. Templates에서 **SvelteKit Boilerplate** 카드 열기.
2. **Deploy** 클릭.
3. 저장소 이름 확인 → **Deploy**.
4. **Ready** → **Visit**.
5. SvelteKit 기본 화면이 보이면 성공.

카드가 안 보이면 Lab 3(Astro)만으로도 “프레임워크 템플릿 원클릭” 연습은 충분.

끝: 배포 URL 확인.

이번 목록에서 제외 (Skip)

항목	이유
Commerce / Shopify / Stripe	결제 · 상점 설정 부담
Auth + DB 풀스택(Supabase 등)	외부 계정·Env 필수
AI Chatbot(키 필수)	API 키 · 과금
Monorepo / Enterprise	초보 원클릭 범위 밖

부록 B를 끝낸 뒤

1. Dashboard에 프로젝트 카드가 보이는지.
2. 각 Lab **Visit** URL을 메모장에 한 줄씩.
3. 다음은 본편 **5~7장**(Preview · Env · 도메인 맛보기) 또는 부록 A.

템플릿은 구경이 아니라, **Deploy 한 번 → URL 한 번**이면 성공.

나두Ai · 부록 B · 가벼운 템플릿 배포 · 무료 전자책

고지 · 법적 주의

※ 이 문서는 안내 체크리스트이며 법률 의견이 아닙니다.

『AI 하는 사람을 위한 Vercel 초간단』 (나두Ai)을 이용하기 전에 한 번 읽어 주세요.

1. 저작권 · 이용 범위

- 이 전자책 PDF의 저작권은 **나두Ai**(익명 발행)에 있습니다.
 - 무료로 제공하며, **개인 학습용**으로 이용할 수 있습니다.
 - **무단 재판매·유료 재배포·타인 명의의 유료 상품화**는 금지합니다.
 - 공식 배포 주소(<https://tinalinkeom.vercel.app/ebook>)를 공유하는 것은 허용합니다.
-

2. 제3자 상표 · 비제휴

- Vercel, GitHub 및 각종 AI·오픈소스·클라우드 명칭은 각 권리자의 상표일 수 있습니다.
 - 이 전자책·나두Ai는 Vercel·GitHub 등의 **공식 교육·제휴·인증 커리큘럼이 아닙니다**.
 - 화면 문구는 서비스 업데이트로 바뀔 수 있으니, 비슷한 위치·비슷한 문구를 기준으로 따라가 주세요.
-

3. 제3자 코드 · 저장소 · 템플릿

- 실습은 공개 저장소·템플릿 Import·배포 안내입니다. 이 PDF가 타인의 소스코드를 상품으로 재판매하지 않습니다.
 - 각 저장소·템플릿의 **라이선스(MIT, Apache 등)**는 독자가 직접 확인·준수해야 합니다.
 - 예시로 든 저장소·템플릿·도메인 안내는 시점에 따라 삭제·변경될 수 있습니다.
-

4. 나두Ai 콘텐츠

- TikTok·YouTube 등 나두Ai 영상은 별도 작품입니다.
 - 이 전자책은 **따라하기 가이드**이며, 영상 파일 자체를 포함·재배포하지 않습니다.
 - 영상은 삭제되거나 노출이 바뀔 수 있습니다.
-

5. 교육 목적 면책

- 학습·실습 안내입니다. 특정 결과·수익·취업·보안·가동률 완벽을 보장하지 않습니다.
 - 도구·API·요금제·화면·DNS·빌드 환경은 변경될 수 있습니다.
 - API 키·비밀번호·`.env` / `.env.local`·Vercel Environment Variables 등 비밀값이 유출되지 않도록 관리할 책임은 독자에게 있습니다. 커밋·공유·스크린샷 전에 한 번 더 확인하세요.
-

6. 금지 안내

불법 스크래핑, 서비스 약관 위반, 무단 침해, 악성 자동화, 타인 계정·도메인 무단 사용 등에 이 자료를 사용하지 마세요.

7. 제공 방식 · 문의

항목	내용
브랜드	나두Ai
상품	『AI 하는 사람을 위한 Vercel 초간단』 PDF (무료)
받는 곳	https://tinalinkeom.vercel.app/ebook
문의	나두Ai 채널(@nadooai) 및 https://tinalinkeom.vercel.app

항목	내용
	(tinalinkeom)

현재는 **무료** 제공됩니다. 유료 판매·환불 규정은 적용하지 않습니다.

나두Ai